Escuela Profesional de Ciencias de la Computación

Curso: Estructuras de Datos Avanzadas

PRÁCTICA DE LABORATORIO 4

Analizador Léxico SQL

Tema:	Especificación de Trabajo Práctico	**Duración:**	2 horas
Lenguaje:	C++	**Modalidad:**	Grupal
Puntaje:	20 puntos		

1. Introducción y Contexto General

El objetivo de este proyecto práctico es la construcción progresiva de un servidor de base de datos relacional para sentencias simples de tipo `SELECT`.

El presente trabajo corresponde a la primera parte de la Fase 2 y se centra en el diseño e implementación del analizador léxico. Este componente constituye la primera etapa del procesamiento de una sentencia SQL. Su función es examinar el texto de entrada, reconocer sus componentes léxicos y generar una secuencia ordenada de tokens que pueda ser utilizada posteriormente por el analizador sintáctico.

2. Objetivo del Trabajo

Diseñar e implementar un analizador léxico para un subconjunto de sentencias SQL de tipo `SELECT`, capaz de transformar una cadena de entrada en una secuencia de tokens correctamente clasificados, proporcionando una interfaz de salida adecuada para su integración con el analizador sintáctico.

3. Planteamiento Técnico

El analizador léxico recibirá como entrada una sentencia SQL representada como una cadena de caracteres. A continuación, examinará sus caracteres, identificará los lexemas y los clasificará según las categorías definidas en la especificación del lenguaje.

Como resultado, generará una secuencia de tokens que conservará el orden en que aparecen en la sentencia original.

Los nombres de los tokens son referenciales. Cada equipo podrá adoptar una nomenclatura diferente, siempre que sea consistente y esté documentada.

El analizador léxico no deberá ejecutar la consulta ni comprobar la existencia de las tablas o columnas. Su responsabilidad consiste en reconocer y clasificar los componentes léxicos de la entrada.

3.1. Categorías de Tokens

Como mínimo, se deberán considerar las siguientes categorías:

– **Palabras reservadas:** `SELECT`, `FROM`, `WHERE`, `AND` y `OR`.

- **Identificadores:** nombres de tablas, columnas y otros elementos admitidos por el subconjunto SQL implementado.
- **Literales numéricos:** números enteros y decimales.
- **Literales de texto:** cadenas delimitadas por comillas simples.
- **Operadores relacionales:** `=`, `<>`, `<`, `>`, `<=` y `>=`.
- **Signos de puntuación:** coma (`,`) y punto y coma (`;`).
- **Fin de entrada:** token especial `EOF`, que indica que no existen más caracteres por procesar.

El equipo deberá documentar las reglas léxicas utilizadas, incluyendo el tratamiento de mayúsculas y minúsculas, identificadores, espacios en blanco, cadenas entre comillas y caracteres no admitidos.

4. Componentes Principales

La implementación deberá contemplar los siguientes componentes:

- **Módulo de lectura de entrada:** recibe la sentencia SQL desde la entrada estándar o desde un archivo.
- **Modelo de tokens:** representa cada token mediante información como su tipo, lexema y posición en la entrada.
- **Reconocedor léxico:** examina los caracteres y determina qué token corresponde a cada secuencia reconocida.
- **Tabla de palabras reservadas:** permite diferenciar las palabras reservadas de los identificadores ordinarios.
- **Manejo de errores:** identifica caracteres no válidos y secuencias léxicas mal formadas, informando su ubicación.
- **Interfaz de salida:** presenta los tokens generados en un formato legible y adecuado para su utilización por el analizador sintáctico.

4.1. Requisitos Funcionales

El analizador léxico deberá cumplir los siguientes requisitos:

1. Leer y procesar una sentencia SQL.
2. Reconocer y clasificar los tokens definidos en la especificación del lenguaje.
3. Mantener el orden original de los tokens.
4. Registrar, como mínimo, la línea y la columna donde comienza cada token, o proporcionar otra representación documentada de su posición.
5. Reconocer operadores de uno y dos caracteres sin dividir incorrectamente los operadores compuestos.
6. Ignorar los espacios en blanco que no sean significativos para el lenguaje.
7. Diferenciar palabras reservadas de identificadores, respetando las reglas definidas por el equipo.
8. Detectar errores léxicos y reportar su ubicación.

5. Especificación de Entregables

Para completar satisfactoriamente el trabajo, se deberán presentar los siguientes componentes:

Entregable	Componentes clave	Criterio de aceptación
1. Modelo de tokens	Tipos de tokens, lexemas y posiciones.	Cada token contiene información suficiente para identificar su categoría y ubicación.
2. Analizador léxico	Algoritmo de exploración y reconocimiento de lexemas.	Reconoce y clasifica correctamente los componentes de las sentencias SQL admitidas.
3. Manejo de errores	Detección de caracteres inválidos y secuencias mal formadas.	Informa los errores léxicos y sus posiciones sin clasificar las secuencias incorrectas como tokens válidos.
4. Pruebas	Pruebas unitarias, casos límite y entradas inválidas.	Verifica los tokens esperados y el comportamiento ante errores léxicos.
5. Documentación y Repositorio	README, especificación de tokens e instrucciones de uso en GitHub.	Permite compilar, ejecutar y probar el proyecto siguiendo las instrucciones documentadas.
6. Informe PDF	Documento formal en formato PDF detallando la arquitectura, modelo de tokens, manejo de errores, pruebas y conclusiones.	Documento estructurado y completo subido al classroom según las instrucciones de entrega.

6. Pruebas y Validación

El equipo deberá diseñar pruebas que permitan verificar el comportamiento del analizador en diferentes situaciones. Como mínimo, se deberán contemplar los siguientes escenarios:

1. **Sentencia válida:** procesar una consulta que incluya las cláusulas `SELECT`, `FROM` y `WHERE`.
2. **Operadores compuestos:** comprobar que `>=`, `<=`, `<>` y `!=` se reconozcan como tokens individuales.
3. **Literales:** verificar el reconocimiento de números enteros, decimales y cadenas de texto.
4. **Palabras reservadas e identificadores:** comprobar que las palabras reservadas se distingan correctamente de los identificadores.
5. **Espacios en blanco:** verificar el procesamiento de espacios, tabulaciones y saltos de línea.
6. **Errores léxicos:** introducir caracteres no admitidos y cadenas sin cerrar para verificar los mensajes de error.
7. **Casos límite:** probar entradas vacías, sentencias incompletas y tokens al final de la entrada.

Los resultados de las pruebas deberán documentarse en el informe PDF y formar parte de la entrega.

7. Entrega en GitHub e Informe

Toda la solución deberá estar alojada y gestionada dentro de un repositorio de GitHub, público o privado con los permisos de acceso correspondientes. El **Informe PDF** deberá ser incluido dentro de la raíz de la entrega del repositorio o cargado junto a los entregables del trabajo.

8. Rúbrica de Calificación

La evaluación se realizará sobre una escala de 0 a 20 puntos, distribuida según los siguientes criterios:

Criterio	**Descripción del desempeño**	**Puntaje**
Reconocimiento de tokens	Clasificación correcta de palabras reservadas, identificadores, literales, operadores y signos de puntuación.	7.0 pts
Algoritmo y diseño	Correcta exploración de la entrada, representación de tokens, tratamiento de posiciones y separación de responsabilidades.	5.0 pts
Manejo de errores y pruebas	Detección de errores léxicos y existencia de pruebas para casos válidos, inválidos y límite.	4.0 pts
Demostración e Informe PDF	Ejecución correcta del programa, demostración de escenarios e informe PDF claro y completo.	4.0 pts
Calificación Total		**20.0 pts**

9. Integración con las Siguientes Etapas

El analizador léxico deberá implementarse como un módulo independiente del resto del motor de base de datos. Su salida será utilizada posteriormente por el analizador sintáctico, responsable de comprobar que la secuencia de tokens cumple la gramática definida para el subconjunto SQL implementado y de generar el árbol de consulta correspondiente.

En consecuencia, el presente trabajo no requiere ejecutar consultas ni acceder al almacenamiento de tablas. Su responsabilidad se limita al reconocimiento léxico y a la generación de una secuencia de tokens correcta y documentada.

10. Consideraciones Finales

El equipo deberá demostrar tanto el correcto funcionamiento del programa como el dominio de los conceptos relacionados con el análisis léxico.

Se valorará especialmente la claridad del diseño, la consistencia en la clasificación de tokens, el tratamiento de errores, la calidad del informe PDF y las pruebas, además de la facilidad para integrar el componente con las etapas posteriores del proyecto.